

Re:Seat

KBO 야구 경기 좌석 예매 플랫폼

3차(최종) 프로젝트 결과 발표 — 성능 실측과 신규 기능

기간

2026.06.25 ~ 09.16

1차~3차, 총 12주

3차 기간

2026.08.04 ~ 09.16

약 6주

팀 구성

TEAM 3조

Back-To-Back · 5인

전윤현(팀장), 김재환, 박현수, 유명인, 조하린

목차

01 프로젝트 개요

프로젝트 소개 · 3차 범위와 우선순위

02 팀 구성 및 역할

도메인별 담당(A~E) · 공통 협업 규칙

03 수행 절차 및 방법

3차 일정 · 진행 순서 · 리스크 관리

04 수행 결과

서비스 흐름 · Admin 페이지 · A~E 도메인 구현

05 테스트 · 검증 결과

Queue · 좌석 선점 · 예매 상태 · 결제 — k6 성능 및 동시성 검증

06 자체 평가 의견

잘한 점 · 아쉬운 점 · 향후 계획

01

프로젝트 개요

프로젝트 소개 · 주제 및 선정 배경 · 기대 효과 · 활용 장비 및 재료

한 줄 정리

공정한 대기 → 안전한 좌석 선점 → 신뢰할 수 있는 결과를 하나의 흐름으로 제공하는 티켓팅 플랫폼

서비스 정의

무엇을 하는가

예매 오픈 시점의 접속 혼잡, 좌석 중복 선점, 암표 거래 문제를 줄여 팬이 정해진 절차 안에서 안정적으로 티켓을 예매하게 한다.

핵심 가치

무엇을 지키는가

순간 트래픽 급증에도 요청을 대기열로 흡수해 안정적으로 처리하고, 대기열과 분산 락으로 1인 1좌석의 정합성을 보장한다.

주요 대상

누구를 위하는가

인기 KBO 경기를 직접 예매해야 하지만 오픈 시점의 혼잡과 빠른 매진으로 반복적으로 실패하는 야구 팬.

기대 효과

- 01 기술 · 락 트레이드오프(성능·데드락·정합성)를 체득하고 분산 락 효과를 수치로 확인
- 02 서비스 · 대기열 우회·중복 예매 제한, 본인인증부터 티켓 발급까지 이어지는 유통 구조

3차 목표 — 신뢰할 수 있는 예매 사이트

1·2차에서 완성한 예매 파이프라인 위에, 3차는 정합성·성능을 수치로 증명하고 운영에 필요한 기능을 채우는 데 집중합니다.

1

1순위

없으면 예매 흐름이 성립하지 않거나 설계 논리·정합성을 증명할 수 없는 기능

1차 스프린트 (8/20)

2

2순위

서비스 완결성과 운영 가시성을 위해 필요한 기능

2·3차 스프린트 (8/27, 9/3)

3

3순위

있으면 좋으나 지연 시 축소·제거 가능한 기능

4차 스프린트 (9/10)

3차 검증 목표

목표	내용
좌석 병렬성	서로 다른 좌석 요청이 전역 락으로 직렬화되지 않음
대기열 공정성 · 결제 정합성	Queue-Token-CAPTCHA 미통과 접근 차단 / 중복 승인 없이 실패·취소·환불 결과 일관
환불 정합성 · 수량 제한	PG 취소 성공 후 DB 반영 실패해도 이중 환불 0건 / 누적 2매 초과 0건
운영 가시성	핵심 흐름의 지연·오류율·처리량·자원 상태를 조회하고 알림을 받을 수 있음

02

프로젝트 팀 구성 및 역할

5인 · 도메인 단위로 나눈 담당 업무

도메인 단위 분업 구조

1차부터 유지해 온 도메인 소유권을 3차에도 그대로 적용해, 각자 자기 도메인의 신뢰성·성능·신규 기능을 책임집니다.

구분	소유 도메인	주요 역할	담당자
A. 인증·사용자·인프라	User, Verification, Infra, Citydata	인증·인가, 회원 관리, 배포·운영 환경, 외부 API(날씨·지도) 연동, 모니터링	김재환
B. 대기열·주문	Queue, Order	진입 게이트, 대기열, SSE, 입장 토큰, 주문 상태·만료	전윤현
C. 경기·좌석·예약	Game, Stadium, Team, SeatInventory, Reservation	경기·좌석 조회, 좌석 HOLD·해제·만료, 동시성 제어와 락 전략 비교	조하린
D. 결제	Payment	Toss Payments, 먹등 승인, 부분·전체 환불, 취소 이력 관리, PG 복구	박현수
E. 티켓	Ticket	티켓 발급·상태 관리, QR 검표, 마이페이지 통합 조회, 관리자 조회·강제 취소	유명인

개발 프로세스 및 협업 규칙

코드 리뷰 · 테스트

전원 참여, PR 승인 2명 이상 후 Squash merge

API 문서 · Swagger

도메인 담당자 작성, 전원 검토

ERD · 시스템 구성도

변경 도메인 담당자 갱신, 전원 검토

프론트엔드

AI 페어 프로그래밍으로 구현, 담당자가 자기 도메인 화면 연동 검토

회의록 · 발표 자료

Notion 공동 관리

도메인 간 계약

구현 착수 전 Swagger·DTO로 확정 후 병렬 개발

브랜치 전략

GitHub Flow — 1이슈=1브랜치=1PR, type/이슈번호-도메인-요약 네이밍

코드 컨벤션

Naver HackDay Convention, Spotless 자동 포맷

03

프로젝트 수행 절차 및 방법

3차 일정 · 진행 순서 · 리스크 관리

수행 일정

PHASE 01

세미 1차

6/25(목) ~ 7/11(토)

- 회원가입 · 로그인, 경기 · 구장 · 좌석 조회
- 대기열 진입과 순번, 좌석 선점, 주문
- Toss 테스트 결제, 예매 흐름 연결

PHASE 02

세미 2차

7/13(월) ~ 7/29(수)

- Kafka 대기열 전환 · SSE 실시간 순번 · 입장 토큰 검증
- 결제 먹등기 · 복구 스케줄러, PortOne 본인인증
- 티켓 발권, 공통 응답 규약 · Swagger 정비
- 동시성 검증 확장

3차 일정 — 2026.08.04 ~ 09.16 (약 6주)

매 스프린트 종료 시점에 버전을 배포해(v0.1.0 → v0.4.0), 발표 직전까지 회귀 위험을 분산시켰습니다.

기간	스프린트	목표	산출물
08.04 ~ 08.08	준비	범위·도메인 계약·API·시간 정책 확정	기획서 v4, Swagger 기준
08.09 ~ 08.25	1차	2차 잔여 부채 마무리, 비활성 테스트 복구, T3-01~08	CI 실행 로그, 버그 리포트, k6 1차 결과
08.26	v0.1.0	티켓 부분 환불·재시도, 결제 부분 취소 이력·복구 구조, 도시데이터·카카오맵 연동, 관리자 예매 상태 전이	GitHub Release Notes
08.27 ~ 09.01	2차	T3-10 모니터링, T3-09 E2E, T3-11 락 3종 비교	E2E 결과, 락 비교 리포트
09.02	v0.2.0	좌석 상태 전이 메서드 정비, RedissonConfig 프로필 분리, 티켓·좌석 도메인 리팩토링	GitHub Release Notes
09.03 ~ 09.08	3차	T3-12~14 관리자·홈·마이페이지 화면	사용자·관리자 화면, 관리자 API
09.09	v0.3.0	v0.2.0에 프론트엔드 개선 반영	GitHub Release Notes
09.10 ~ 09.13	4차·버퍼	전체 회귀·부하 재측정, 시연 리허설	성능 리포트, 발표 자료 초안
09.15	v0.4.0	관리자 티켓(QR 검표·재발급·일괄 취소), 대기열 지표, 좌석 재고 요약·차단/해제, 취소·재선점 UNIQUE 제약 재설계, 결제·대기열 성능 테스트 스크립트 추가	GitHub Release Notes
09.14 ~ 09.16	마무리	시연 리허설 최종, 발표 자료 확정	발표 자료, 시연 영상, 최종 발표

3차 구현 기능 (T3-01 ~ T3-16)

이후 슬라이드에서 도메인별로 하나씩 상세 결과를 다룹니다.

ID	기능	담당
T3-01	예매 흐름 원자성 보강 (Queue-Token 검증만, 선점 실패 시 상태 복구)	B, C
T3-02 · T3-03	예매 자격·수량 게이트 / 좌석 취소·재선점 흐름	C, D, E
T3-06 · T3-07	부분 환불·티켓 상태 전이 / PG 상태 연동 복구	D, E
T3-08 · T3-09	사용자 티켓·입장(QR) 연동 / 실제 E2E 연동	E, 공통
T3-10	Prometheus/Grafana 모니터링	A
T3-11	락 전략 3종 비교	C
T3-12	도메인별 관리자 페이지	AI(FE) + 각 도메인
T3-13	홈·마이페이지 통합 조회 API 및 화면	A, E, AI(FE)
T3-14	날씨·카카오 지도 정보	A, C

04

프로젝트 수행 결과

서비스 흐름 · 관리자 페이지

예매 파이프라인 — 경기 선택부터 티켓 발권까지 6단계

6단계 흐름을 화면 단위로 짚어보며, 각 단계에서 어떤 기술적 장치가 함께 동작하는지 이어서 설명합니다.

1

경기 선택

QueryDSL 동적 검색으로 구단·구장·날짜별 경기 조회,
예매 상태(OPEN/CLOSED)에 따라 진입 가능 여부 표시

2

대기열 진입

Kafka로 진입 이벤트 발행 → Redis ZSET 등록
→ SSE로 순번·입장 상태 실시간 수신

3

좌석 선택·선점

입장 토큰 보유자만 좌석 조회 가능, 구역별 배치도에
서 최대 2석 선택 → Redisson 분산락으로 선점

4

주문 생성

선점된 예약 기준 주문 생성, 결제 제한시간 부여,
시간 초과 시 자동 취소 후 좌석 반환

5

결제

역등키와 함께 결제 요청 생성 → Toss 결제창
→ 콜백 검증 후 승인, 실패 시 복구 스케줄러 재처리

6

티켓 발권

결제 승인 시 좌석 SOLD 확정 및 티켓 발급,
마이페이지에서 티켓 기준 통합 조회

STEP 1 · 경기 선택 — 오늘의 경기와 예매 캘린더

달력에서 날짜를 고르면 해당 일자의 경기 목록이 카드로 표출되고, 예매 상태에 따라 진입 가능 여부가 즉시 구분됩니다.

2026 KBO LEAGUE

지금 바로 예매하세요

KBO 리그 전 구단 홈 경기를 확인하고
공정한 대기열을 통해 원하는 좌석을 선택하세요.

TODAY'S GAME · 예매중

롯데 자이언츠 vs 삼성 라이온즈
9월 15일 (화) 18:30 · 대구삼성라이온즈파크

예매하기 →

● 예매 예정 ● 예매중 ● 예매 종료 ● 경기 취소

2026.09.15 경기 4개 일정 날짜 선택 해제

9월 15일 화

예매중

삼성 라이온즈
VS
롯데 자이언츠

롯데 자이언츠 vs 삼성 라이온즈
9월 15일 (화) 18:30 · 대구삼성라이온즈파크 · 지금 예매할 수 있습니다.

예매하기 →

9월 15일 화

예매중

SSG 랜더스
VS
KIA 타이거즈

KIA 타이거즈 vs SSG 랜더스
9월 15일 (화) 18:30 · 인천SSG랜더스필드 · 지금 예매할 수 있습니다.

예매하기 →

9월 15일 화

예매중

NC 다이노스
VS
LG 트윈스

LG 트윈스 vs NC 다이노스
9월 15일 (화) 18:30 · 창원NC파크 · 지금 예매할 수 있습니다.

예매하기 →

9월 15일 화

예매중

한화 이글스
VS
KT 위즈

KT 위즈 vs 한화 이글스
9월 15일 (화) 18:30 · 대전 한화생명 불펜파크 · 지금 예매할 수 있습니다.

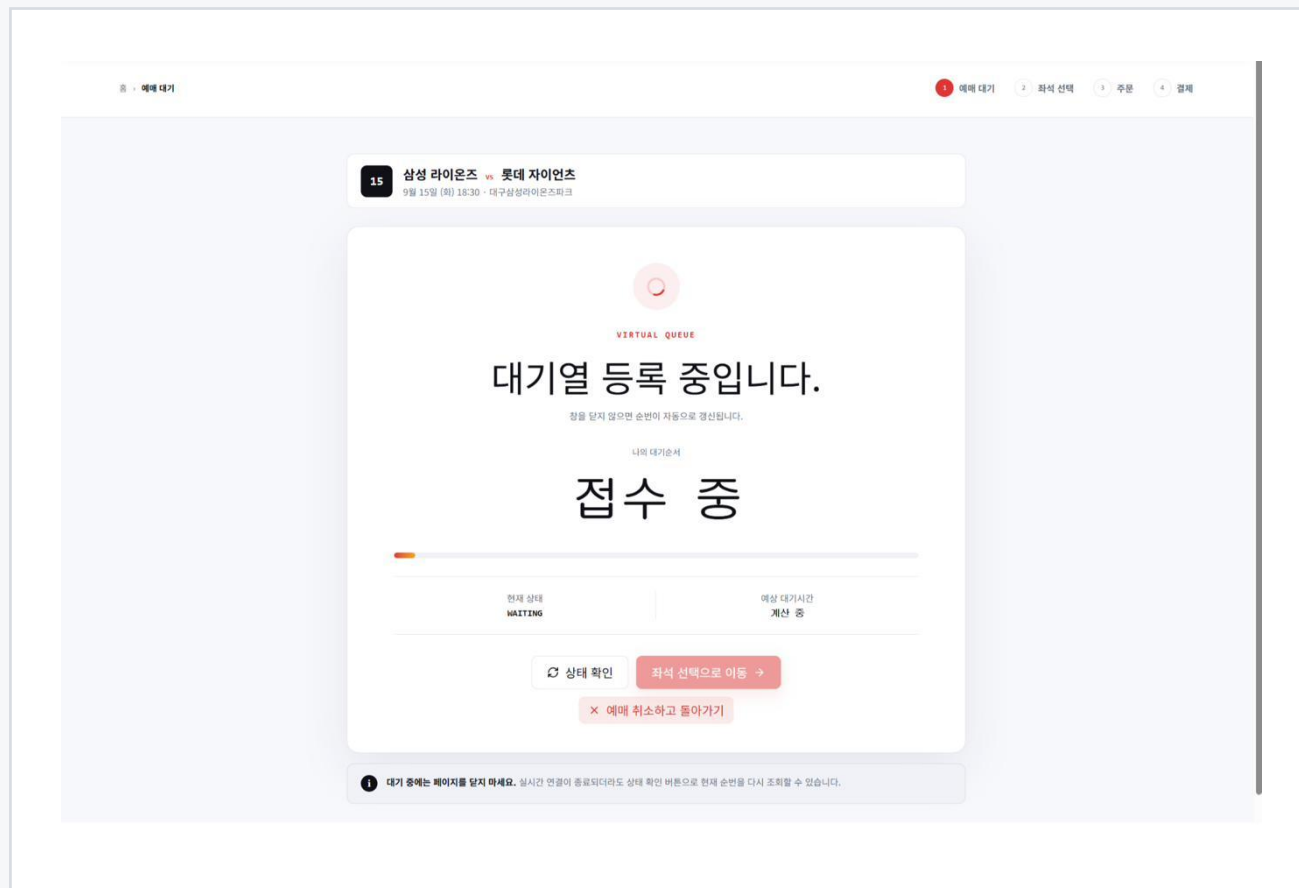
예매하기 →

화면 설명 캘린더의 날짜별 점(●) 개수로 경기 수를 한눈에 보여주고, 날짜를 선택하면 해당일 전체 경기가 카드 목록으로 펼쳐집니다. 각 카드는 예매 예정/예매중/예매 종료/경기 취소 상태를 색으로 구분해 보여줍니다.

기술 포인트 QueryDSL 동적 쿼리로 구단·구장·날짜 조건을 조합해 검색하고, 경기별 bookingStatus(OPEN/CLOSED)에 따라 “예매하기” 버튼의 활성 여부를 서버 응답 기준으로 제어합니다.

STEP 2 · 대기열 진입 — Virtual Queue

경기를 선택하면 대기열 등록 화면으로 전환되고, 접수 상태와 예상 대기시간이 실시간으로 갱신됩니다.



화면 설명

“대기열 등록 중입니다” 화면에서 나의 대기 순서, 현재 상태(WAITING), 예상 대기시간을 보여주며, 창을 닫아도 순번은 서버에서 계속 관리됩니다.

기술 포인트

Controller가 요청을 즉시 202 Accepted로 응답하고 Kafka에 진입 이벤트를 비동기 적재합니다.

Consumer가 Redis ZSET에 순번을 등록한 뒤, 스케줄러가 상위 N명에게 입장을 허용하고 SSE로 순번·입장 상태를 클라이언트에 실시간 푸시합니다.

STEP 3 · 좌석 선택·선점 — 구역 선택부터 확정까지

입장 토큰을 받은 사용자만 좌석 배치도에 접근할 수 있고, 구역과 좌석을 고르면 남은 선점 시간이 함께 표시됩니다.

01 구역 선택
원하는 구역을 먼저 선택하세요.

101 내야 · 18,000원 50 / 50석	102 내야 · 18,000원 50 / 50석
103 내야 · 18,000원 50 / 50석	301 내야 · 18,000원 50 / 50석
302 내야 · 18,000원 50 / 50석	303 내야 · 18,000원 50 / 50석
201 외야 · 16,000원 50 / 50석	202 외야 · 16,000원 50 / 50석
203 외야 · 16,000원 50 / 50석	204 외야 · 16,000원 50 / 50석

02 좌석 선택
101의 실제 좌석을 선택하세요.

그라운드 방향

A 1 2 3 4 5 6 7 8 9 10

03 선택 확인
최대 2석까지 선택할 수 있습니다.

좌석 선택 남은 시간 **1:40**

101 A열 9번	21,600원
101 A열 10번	21,600원
총 결제 예정 금액	43,200원

2석 선점하기 →

입장 후 3분 안에 처음 좌석을 선점해야 하며, 선점 후에는 예약 만료시간이 적용됩니다. 선점 해제 후에는 최초 선점 기준 재선점 가능 시간 안에 다시 선점할 수 있습니다.

화면 설명

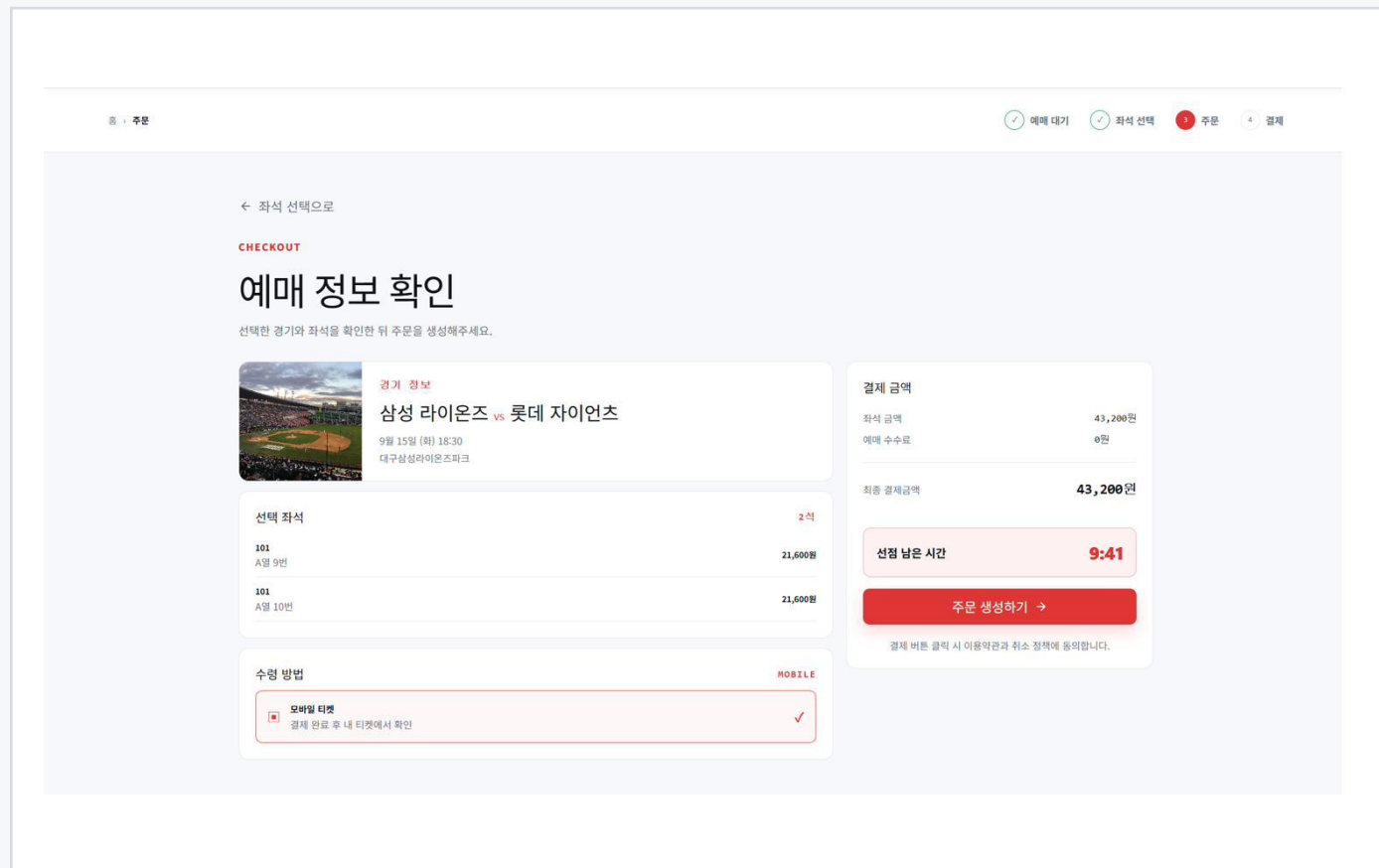
01 구역 선택 → 02 좌석 선택 → 03 선택 확인 3단계로 구성되며, 선택한 좌석과 함께 금액이 우측 패널에 실시간으로 반영되고 "좌석 선택 남은 시간" 카운트다운이 함께 노출됩니다.

기술 포인트

Redisson 분산락으로 좌석 ID 단위 선점을 보장하고 GameSeatStatus를 AVAILABLE → HELD로 전이합니다. 다좌석 선점 시 gameId 오름차순 정렬로 락을 획득해 순환 대기(deadlock)를 구조적으로 차단합니다.

STEP 4 · 주문 생성 — 예매 정보 확인

선택한 좌석과 결제 금액을 확인하는 주문서 화면으로 이동하며, 결제 제한시간이 함께 표시됩니다.



화면 설명

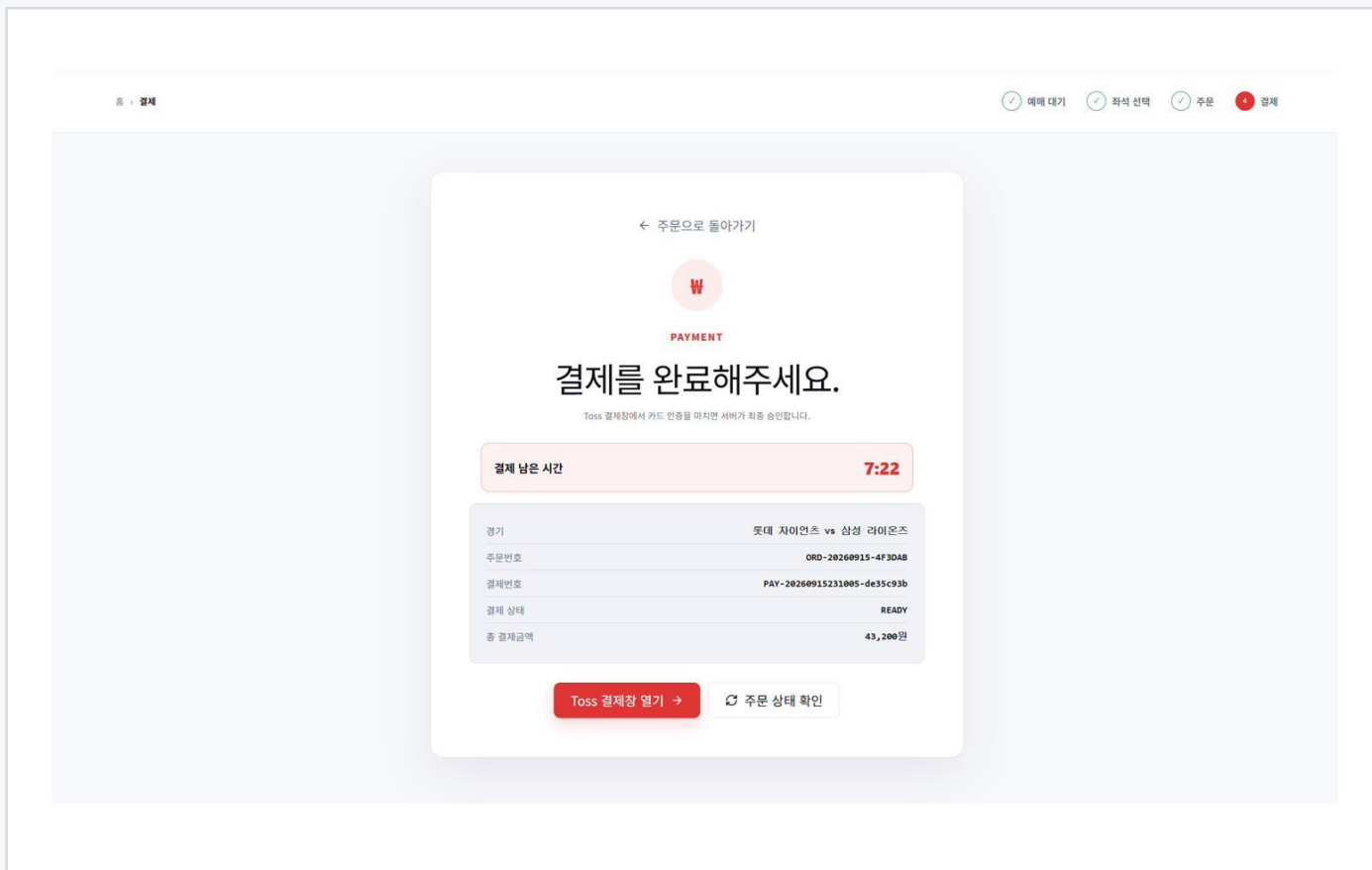
경기 정보, 선택 좌석, 결제 금액, 수령 방법(모바일 티켓)을 한 화면에서 확인하고 “주문 생성하기” 버튼으로 다음 단계로 넘어갑니다.
“선점 남은 시간” 타이머가 계속 노출됩니다.

기술 포인트

선점된 예약(Reservation)을 기준으로 주문(Order)을 생성하고 결제 마감시각(payment_deadline)을 부여합니다.
시간 초과 시 스케줄러가 주문을 자동 취소하고 좌석을 즉시 반환합니다.

STEP 5 · 결제 — Toss Payments 연동

Toss 결제창으로 이동해 결제수단을 선택하고 결제를 완료하면, 서버가 최종 승인 여부를 확정합니다.



화면 설명

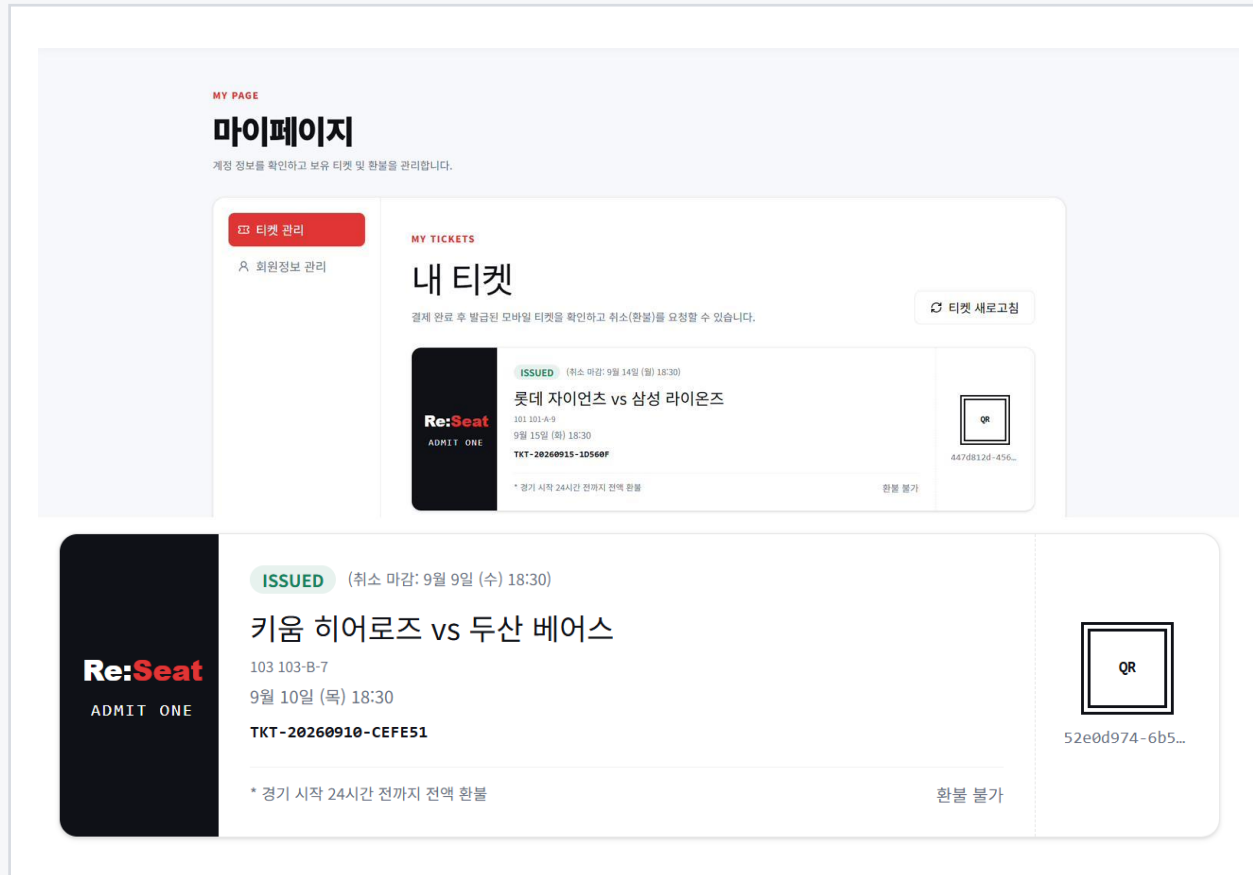
주문번호·결제번호·결제 상태(READY)와 남은 결제 시간을 보여주고, "Toss 결제창 열기"로 실제 결제를 진행합니다. "주문 상태 확인"으로 진행 상황을 다시 조회할 수 있습니다.

기술 포인트

멧등키(Idempotency-Key)를 포함해 결제 요청을 생성하고, paymentKey·orderId·amount 3중 대조로 콜백을 검증합니다. 승인 결과가 불명확하면 복구 스케줄러가 PG를 재조회해 최종 상태를 맞춥니다.

STEP 6 · 티켓 발권 · 마이페이지 통합 조회

결제가 승인되면 예매 완료 화면과 함께 티켓이 즉시 발권되고, 마이페이지에서 QR 티켓과 예매 내역을 한 화면에서 확인합니다.



화면 설명

마이페이지 “내 티켓” 탭에서 티켓 상태(ISSUED), 취소 마감일, 좌석·경기 정보, QR 코드를 함께 보여주고, 결제·환불 상세도 같은 화면에서 조회할 수 있습니다.

기술 포인트

결제 승인 시 좌석을 SOLD로 확정하고 티켓을 발급합니다.

GET /api/v1/tickets/me가 경기·좌석·주문·결제·환불 정보를

티켓 기준으로 통합 조회하며, group 필터로 예매확인/예매취소 탭을 구분합니다.

관리자 페이지 — 도메인별 운영 기능을 하나의 화면 체계로

각 도메인 담당자가 자신의 관리자 API를 구현하고, 프론트는 AI 페어 프로그래밍으로 화면을 통합했습니다.

담당	관리자 기능
A. 인증·사용자	회원 이메일·이름·권한(일반/관리자)·상태(활성/정지/탈퇴) 관리
B. 대기열·주문	현재 대기 인원·Queue-Token 발급 지표 조회, 입장 허용 인원 실시간 조절, 일/주/월 집계
C. 경기·좌석·예약	구장·구단·좌석구역·좌석 등록, 경기 등록·재고 오픈, 예약·선점 상태 목록, 좌석 판매 차단
D. 결제	결제 승인·실패·취소 내역 조회, PG 복구 작업 관리
E. 티켓	QR 검표·재발급, 회원별 티켓 조회, 사유 기반 강제 취소, 경기 단위 일괄 취소

다음 페이지부터 경기·좌석 관리 → 대기열 현황 → 티켓 관리 화면을 순서대로 확인합니다.

경기를 검색해 좌석 재고를 생성하고, 판매 가능/선점/판매 완료/차단 현황을 한 화면에서 관리합니다.

SELECTED GAME

롯데 자이언츠 vs 삼성 라이온즈

대구삼성라이온즈파크 · 현재 OPEN

변경할 예매 상태

OPEN

상태 변경 사유

윤영 사유를 입력하세요

예매 상태 변경

좌석 재고 생성

경기 좌석 재고가 아직 생성되지 않은 경우에만 실행하세요.

좌석 재고 생성

좌석 재고

좌석 상태

전체

판매 가능

500

선점

0

판매 완료

0

차단

0

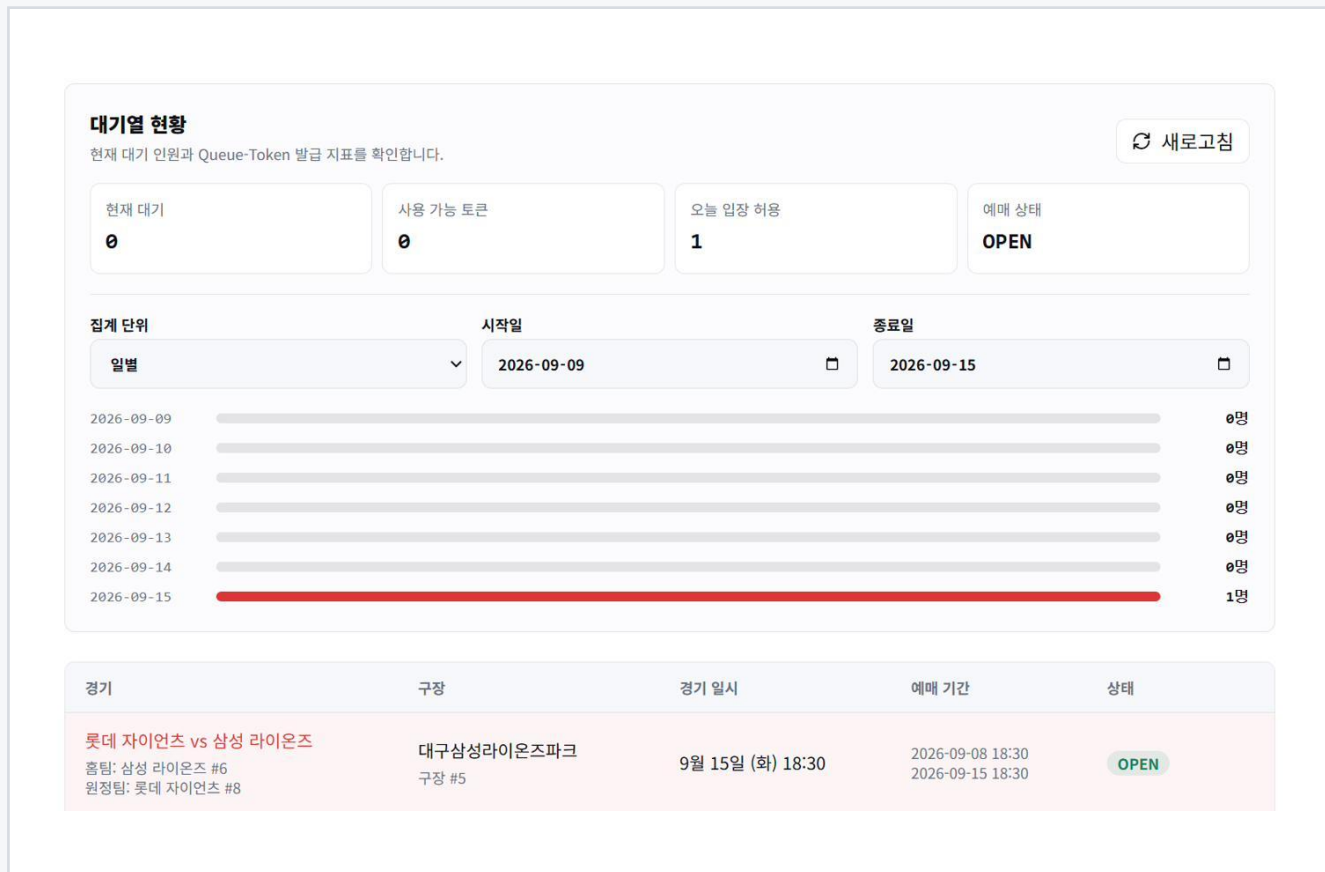
101 · A열 1번 판매 가능	101 · A열 10번 판매 가능	101 · A열 2번 판매 가능	101 · A열 3번 판매 가능	101 · A열 4번 판매 가능	101 · A열 5번 판매 가능
101 · A열 6번 판매 가능	101 · A열 7번 판매 가능	101 · A열 8번 판매 가능	101 · A열 9번 판매 가능	101 · B열 1번 판매 가능	101 · B열 10번 판매 가능
101 · B열 2번 판매 가능	101 · B열 3번 판매 가능	101 · B열 4번 판매 가능	101 · B열 5번 판매 가능	101 · B열 6번 판매 가능	101 · B열 7번 판매 가능
101 · B열 8번 판매 가능	101 · B열 9번 판매 가능	101 · C열 1번 판매 가능	101 · C열 10번 판매 가능	101 · C열 2번 판매 가능	101 · C열 3번 판매 가능
101 · C열 4번 판매 가능	101 · C열 5번 판매 가능	101 · C열 6번 판매 가능	101 · C열 7번 판매 가능	101 · C열 8번 판매 가능	101 · C열 9번 판매 가능

선택한 경기의 예매 상태를 변경하고, 좌석 재고를 상태별(판매 가능·선점·판매 완료·차단)로 집계해 개별 좌석 단위까지 조회·차단할 수 있습니다.

POST /api/v1/admin/games, .../games/{id}/seats로 재고를 열고 bookingStatus를 전
이합니다. 좌석 판매 차단 API는 차단 사유를 함께 기록해 이력을 남깁니다.

대기열 현황 — 실시간 지표와 입장 인원 조절

현재 대기 인원과 Queue-Token 발급 지표를 실시간으로 확인하고, 입장 허용 인원을 관리자가 직접 조절합니다.



화면 설명

현재 대기·사용 가능 토큰·오늘 입장 허용·예매 상태를 카드로 보여주고, 집계 단위(일/주/월)와 기간을 지정해 일자별 대기열 추이를 그래프로 조회합니다.

기술 포인트

입장 허용 인원 조절은 다음 admission 배치 스케줄러 실행 시점부터 즉시 반영되며, 대기열 로그(queue_entry_histories)를 기준으로 집계 API가 응답합니다.

티켓 관리 — QR 검표부터 강제 취소까지

QR 검표·재발급, 회원별 티켓 상태 확인, 사유 기반 강제 취소와 경기 단위 일괄 취소를 한 화면에서 처리합니다.

ADMIN 관리자 페이지
회원과 경기 운영 상태를 관리합니다.

회원 관리
경기/좌석 관리
티켓 관리

TICKETS 티켓 관리
티켓을 검색하고 QR 검표·재발급과 취소 작업을 처리합니다.

QR 검표
경기 ID와 QR 토큰을 확인해 입장을 처리합니다.

경기 ID
QR 토큰
QR 검표

경기 티켓 일괄 취소
해당 경기의 발급 상태 티켓을 모두 취소 접수합니다.

경기 ID
일괄 취소 사유
일괄 취소

회원 ID 전체 상태 연도-월-일 연도-월-일
조회

키움 히어로즈 vs 두산 베어스
TKT-20260910-CEFE51 · 103 103-B-7 · ISSUED
QR 재발급 직권 취소

화면 설명

경기 ID·QR 토큰으로 검표를 처리하고, 회원 ID·상태·기간으로 티켓을 검색해 개별 직권 취소 또는 경기 단위 일괄 취소를 실행합니다.

기술 포인트

강제 취소 시 작업자·사유를 이력으로 남기고, REFUND_FAILED 티켓은 별도 목록에서 수동 처리 상태를 추적할 수 있도록 분리했습니다.

A. 인증·사용자·인프라 — 기반을 다지고 외부 연동을 확장

2차에서 남은 안정성 이슈를 정리한 뒤, 3차에는 모니터링과 외부 정보 연동을 신규로 추가했습니다.

2차 마무리

CI/CD 테스트 환경 구성, ci.yml Kafka 서비스 추가, Spotless 포맷 통일, 시크릿 하드코딩 제거, User 엔티티 중복 메서드·예외 처리 정합성 수정

모니터링 T3-10

Prometheus·Grafana를 AWS Docker 컨테이너에 배포하고 대기열·좌석·주문·결제 핵심 지표를 수집·시각화

외부 연동 T3-14 · T3-16

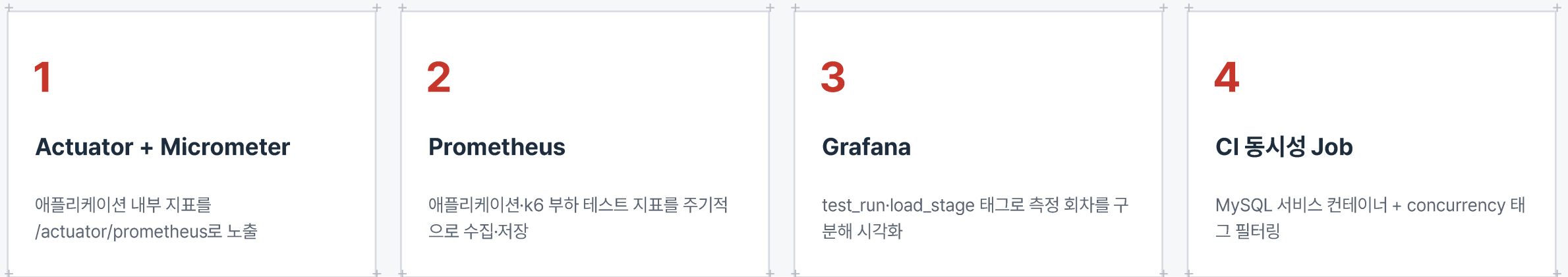
날씨 API 연동 및 혼잡도 지도 오버레이 구현, Linux cron 기반 교통 정체 감지·비상 안내

정책 확장

회원 탈퇴 방지 조건을 ISSUED·REFUND_PENDING·REFUND_FAILED 티켓 보유로 확장

Prometheus / Grafana — 흐름 전체를 관측 가능하게

모니터링은 다른 3차 기능보다 먼저 연결해, 이후 기능 변경의 전후 비교 기준선으로 활용했습니다.



모니터링 대상 비기능 요구사항 (발체)

구분	요구사항	목표
성능	대기열 활성 구간의 p99, 오류율, 단위 시간당 입장 인원	500 VU p99 ≤ 2초, 5xx < 1%
좌석 반환 지연	결제 실패·취소 후 좌석이 AVAILABLE로 복귀하는 지연	즉시 반환 p99 ≤ 1초 / 만료 스케줄러 p99 ≤ 12초

B. 대기열·주문 — 진입을 더 촘촘하게 통제

Queue-Token 소비 시점 변경(계약 v4)을 코드에 반영하고, CAPTCHA·진입 게이트로 자동화된 접근을 추가 차단했습니다.

Queue-Token 정책 반영 T3-01

좌석 선점 시 토큰을 검증만 하고 소비하지 않도록 변경 — 소비 시점을 티켓 발급 시점으로 이관

완료 기준 선점 실패 후 잔여 TTL 안에서 동일 토큰으로 재선점 가능, 부분 성공 0건

CAPTCHA 검증 T3-04

대기열 통과와 좌석 조회·선점 사이에 CAPTCHA 적용

완료 기준 사용자·경기·Queue-Token에 연결된 검증 결과의 만료·위조·재사용 차단

대기열 진입 게이트 T3-05

경기 선택 시점에 순번 없는 1차 진입 게이트 적용, 거절 응답 규격화

완료 기준 게이트 적용 전후 진입 API p99·오류율 비교로 완충 효과 수치화

C. 경기·좌석·예약 — 자격 검증과 취소 이후 흐름 완성

누가, 몇 좌석까지 선점할 수 있는지를 서버가 강제하고, 취소된 좌석이 안전하게 다시 예매 가능해지도록 흐름을 완성했습니다.

예매 자격·수량 게이트 T3-02

본인 인증 여부와 누적 보유 좌석 수(HOLDING 예약 + ISSUED/REFUND_PENDING/REFUND_FAILED 티켓)를 좌석 선점 API에서 검증

완료 기준 미인증 선점 403 차단 100%, 1좌석씩 반복 선점 포함 누적 2매 초과 0건

좌석 취소·재선점 흐름 T3-03

예약·티켓 취소 시 좌석 반환 후 즉시 재선점 가능 상태로 전환, 재선점 시 HOLD 상한 보장 규칙 적용

완료 기준 취소·재선점 동시 요청에서 수량 초과 0건, 좌석 상태 불일치 0건

D. 결제 — 부분 환불과 PG 상태 기반 복구

일부 티켓만 취소해도 금액·좌석이 정확히 맞아떨어지도록, PG 응답이 불명확해도 최종 상태가 항상 일치하도록 만들었습니다.

부분 환불 T3-06

PaymentService.cancelPartial(paymentId, ticketId, cancelAmount) 구현, payment_cancels 엔티티(PENDING/DONE/FAILED) 신설, 취소 이력을 PG 호출 전에 선기록

완료 기준 선택 티켓만 취소되고 나머지는 유지, 사용·환불된 티켓의 중복 요청은 DB 단계에서 차단

PG 상태 연동 복구 T3-07

승인 불명확(CONFIRM_UNCERTAIN)·취소 불명확(CANCEL_UNCERTAIN) 구분, PENDING 잔류 행 복구 스케줄러

완료 기준 PG 재조회 결과로 관련 도메인을 먹등하게 복구, 재시도·실패 이력 기록

결제·주문 상태 확장

orders에 PARTIALLY_CANCELED 추가, payments.canceled_amount 누적으로 전이 판정

E. 티켓 — 상태를 세분화하고, 흩어진 정보를 한 화면으로

환불 중과 환불 실패를 구분해야 좌석 반환 시점을 정확히 판단할 수 있어, 티켓 상태를 6종으로 분리했습니다.

티켓 상태 6종 전이

ISSUED → REFUND_PENDING → REFUNDED(환불 성공) / → REFUND_FAILED →
REFUND_PENDING(재시도) / → USED_ENTERED(QR 검표) / → USED_NO_SHOW(배치)

마이페이지 통합 조회 T3-13b

GET /api/v1/tickets/me — 티켓 기준으로 경기·좌석·주문·결제·환불 정보 조합, group 필터로 탭 구분

완료 기준 B·C·D가 조회 전용 메서드 제공, E가 조합, A가 회원정보 제공

관리자 티켓 기능 T3-08, T3-12

- QR 검표·재발급
- 회원별 티켓 상태 확인
- 사유 기반 강제 취소 (작업자·사유 기록)
- 일괄(bulk) 취소
- REFUND_FAILED 티켓 목록 조회·수동 처리

홈 · 마이페이지 화면 통합 및 실시간 혼잡도 안내

AI 페어 프로그래밍으로 프론트엔드를 구현하고, 서울시 실시간 도시데이터를 연동해 구장 주변 혼잡도를 안내합니다.

T3-13a 홈 통합 조회 당일 경기·날씨·구장 정보를 서버가 집계해 단일 응답 제공, 캐시·타임아웃·폴백은 서버 책임	T3-13c 홈·마이페이지 화면 홈에서 예매로 이동, 마이페이지에서 예매 생명주기 조회·취소. 지정된 API만 호출	T3-14 연계 실시간 혼잡도 잠실야구장 주변 게이트·지하철·먹거리 구역을 4단계(여유~붐빔)로 안내, 동선 팁 제공
--	--	---

혼잡도 정보 구성 (A 외부 호출 + C 구장 좌표)

구역 분류: 출입구 · 대중교통 · 먹거리/주차

혼잡도 4단계(여유/보통/약간 붐빔/붐빔)와 게이트별 대기시간, 우회 경로를 함께 안내

05

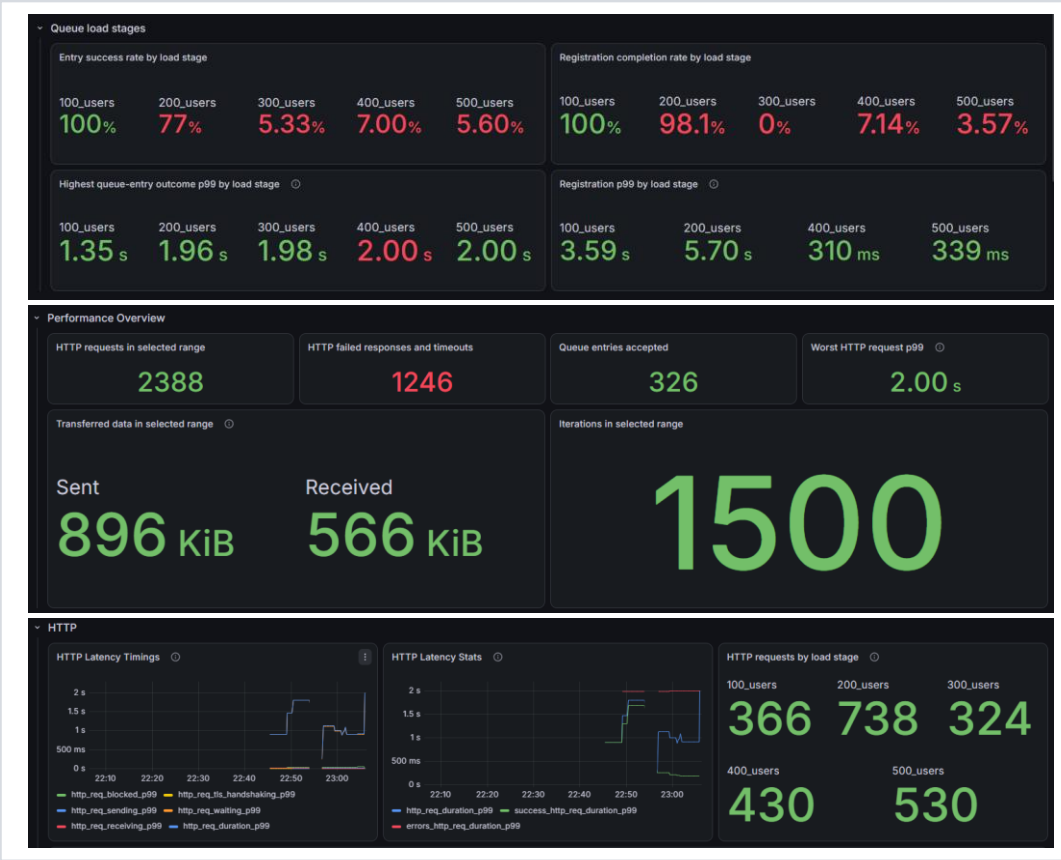
테스트 · 검증 결과

Queue · 좌석 선점 · 예매 상태 · 결제 — k6 성능·동시성 검증

[Queue] 동시 진입 · 등록 완료 — EC2 성능 테스트 결과 (k6)

EC2 배포 환경에서 100→500명까지 순차 부하를 걸어, 접수(202)와 실제 등록 완료를 분리 측정했습니다.

● 측정 완료 — EC2 Ubuntu · RunId queue-ec2-20260915-134218 · 테스트 경기 ID 378 · 서버 5xx 0건



VU	202 접수 성공률	접수 후 등록 완료율	접수 p99	서버 5xx
100명	100%	100%	1.35초	0건
200명	77%	98.1%	1.96초	0건
300명	5.33%	0%	1.98초	0건
400명	7.00%	7.14%	2.00초	0건
500명	5.60%	3.57%	2.00초	0건
합계(1,500회)	21.73%	77.91%	—	0건

결과 해석

300명부터 접수 성공률·등록 완료율이 급락하지만 서버 5xx는 0건 — 실패는 2초 응답 제한 안에 못 들어온 네트워크 오류가 중심입니다. 500명 구간에서 HikariCP pending 185가 관측돼 DB 커넥션 대기 정황이 있습니다(연속 수집 아님, 병목 단독 확정은 보류).

[Reservation] 좌석 선점 분산락 — 동시성 테스트 결과

락을 적용하지 않은 기준선(Before)과 Redisson 분산락 적용 후(After)를 수치로 대조했습니다.

● 측정 완료 — Before(락 미적용) → After(분산락) → 전체 회귀(reservation.service 16개 테스트 100%)

구분	선점 성공	차단 방식	DataIntegrityViolation	LOCK_FAILED
Before · 락 미적용	1건 (우연히 직렬화)	DB 유니크 제약(최후 방어선)	1건	—
After · Redisson 분산락	1건	SEAT_ALREADY_HELD(409) 서비스 레벨 차단	0건	0건
After · 교차 순서 다좌석	1건	15초 내 데드락 없이 종료	0건	1건 (정상 경합)

핵심 발견 — 서비스 레벨 차단 부재의 위험

Before는 서비스 레벨 차단이 없어 나머지 9건이 DB 유니크 제약에서야 막혔고, 이는 사용자에게 500 에러가 노출될 수 있는 구조였습니다.
After는 Redisson 분산락으로 gameSeatId 오름차순 정렬 후 락을 획득해 데드락 순환 대기를 차단하고, 초과 요청을 409(SEAT_ALREADY_HELD)로 명시적으로 반환합니다.
reservation.service 패키지 전체 회귀 16개 테스트 100% 통과로 재확인했습니다.

[Reservation] 사용자·경기 단위 락 — 동시성 통합 테스트

동일 사용자가 서로 다른 좌석 3개에 동시 요청해도 누적 보유 상한(2매)이 지켜지는지 검증했습니다.

● 측정 완료 — 좌석 단위 락으로 방어 못한 지점을 사용자·경기 단위 락으로 서비스 레벨 차단

항목	수치	판정
선점 성공	2	✓ 상한만큼만 성공
MAX_SEAT_COUNT_EXCEEDED	1	✓ 초과 요청 정확히 차단
LOCK_FAILED	0	타임아웃 없음
예상 외 예외	0	✓ 데드락 없음
reservation_seats 행 수	2	✓ 성공 건수와 일치

핵심 발견 — 좌석 단위 락만으로는 못 막는 경합

좌석 단위 락은 “같은 좌석” 경합만 막을 뿐, “같은 사용자가 서로 다른 좌석에서” 보유 상한을 우회하는 경로는 방어하지 못합니다.
사용자·경기 단위 락을 추가해 수량 검증을 정상적으로 직렬화했고, 선점 성공 건수(2)와 DB 행 수(2)가 일치해 over-booking 0건을 확인했습니다.

[Reservation] 좌석 선점 분산락 — 안정성 검증 결과 (k6)

k6로 4가지 부하 시나리오(A~D)를 로컬(3회 반복)·AWS(1회)에서 측정해, 반복 실행과 실제 배포 환경에서도 정합성이 유지되는지 확인했습니다.

● 측정 완료 — k6 시나리오 A~D · 서버 5xx 전 구간 0% · over-booking 전 구간 0건

시나리오	목적	로컬 TPS(평균)	AWS TPS	409(정책적) 비율
A · 동일 좌석 경합	전역 락 아님 재증명	47.07/s	93.89/s	98%
B · 다른 좌석 병렬	좌석 단위 락 검증	66.97/s	66.32/s	0%
C · 인기 좌석 혼합	현실적 경합 패턴 재현	86.34/s	49.61/s	4~6% / 2%
D · 예매 오픈 스파이크	HikariCP 풀 고갈 지점 관찰	251.34/s	151.03/s	92.3%

B > A: 전역 락이 아님을 재확인

동일 VU 50명 기준 로컬 3회 전부 시나리오 B(병렬)가 A(경합)보다 처리량이 높았습니다. 좌석 단위 락(lock:game-seat:{id})이 전역 락으로 직렬화되지 않음을 확인했습니다.

D: HikariCP 풀(크기 10) 고갈 재현

도착률 300→600 req/s 구간에서 로컬·AWS 동일하게 약 25초간 풀이 상시 소진되고 pending이 최대 190건까지 누적됨을 확인 — 동일 부하 프로파일에서 재현 가능한 병목 지점.

단위·통합 테스트 계층(B1·B2 대량 반복 100스레드×10회, B4 Redis-DB 정합성 회귀)의 over-booking 0건 검증은 앞선 “좌석 선점 분산락 — 동시성 테스트 결과” 슬라이드에서 함께 확인했습니다.

[Reservation] 좌석 선점 부하 스파이크 — 시나리오 D 상세 분석 (k6)

예매 오픈 순간 급격한 유입에서 지연이 어떻게 열화되는지와 “실패”로 보이는 수치의 실제 구성을 분해했습니다.

● 측정 완료 — 로컬 1회 · AWS 1회 (반복 측정 아님, 목적은 over-booking 검증이 아니라 순수 부하 관찰)

지표	로컬	AWS	차이
http_reqs	10,054건 (251.34/s)	6,480건 (151.03/s)	로컬이 더 많은 요청 처리
http_req_duration avg	1.78초	3.78초	AWS가 2배 이상 느림
hold_success	500	496	유사
hold_seat_already_held	2,950 (29%)	2,108 (33%)	좌석 경합 차단
hold_max_seat_count_exceeded	6,604 (66%)	3,876 (60%)	fixture 상한 정책 거절
dropped_iterations	2,945	6,519	AWS에서 더 많이 드롭

측정 한계 — 반복·규모 확장은 범위 밖

이번 측정은 로컬·AWS 각 1회뿐이라 반복 편차는 확인하지 못했습니다.
fixture 규모 확장은 T3-11에서 검토 예정이며, 회차 간격이 10분을 넘으면 재선점 상한이 결과를 오염시킬 수 있어 회차를 지체 없이 이어 붙여 측정했습니다.

HikariCP(풀 10)는 로컬·AWS 모두 300→600 req/s 구간에서 약 25초간 소진, pending 최대 190건 — AWS(t2.medium) 지연 열화는 인스턴스 사양 차이로 추정.

[Reservation] 예매 상태 전이 및 오픈 — 원자성 테스트

동일 경기에 관리자 10명이 동시에 예매 오픈을 요청해도 정확히 1건만 성공하는지 검증했습니다 (T3-01).

● 측정 완료 — GameRepository.compareAndSetBookingStatus 조건부 UPDATE(CAS) 검증

항목	수치	판정
동시 요청 스레드	10	
성공(1 반환) 건수	1	✓
실패(0 반환) 건수	9	✓
예외 발생 건수	0	✓ 우연한 통과 아님
최종 booking_status	OPEN	✓ over-booking 미발생
테스트 소요 시간	1.079초	CI 부담 낮음

핵심 발견 — 도메인 가드로 못 막는 lost update를 CAS로 차단

두 트랜잭션이 동시에 SCHEDULED 상태를 읽고 둘 다 통과해버리는 lost update는 도메인 가드(Game.openBooking())만으로는 막을 수 없습니다. WHERE절에 현재 상태(SCHEDULED)를 함께 검사하는 조건부 UPDATE로 전환해 DB 레벨에서 정확히 1건만 통과시켰고, 나머지는 예외가 아닌 0건 반환으로 실패를 구분했습니다.

[Reservation] 좌석 취소·재선점 — 동시성 테스트

취소·해제·재선점 흐름(T3-03)의 경합 지점 4곳을 실제 서비스 경로로 각각 검증했습니다.

● 측정 완료 — ReservationService.cancel()·releaseHold(), SeatHoldFacade.holdSeats() 실제 경로 검증

검증 시나리오	동시 스레드	핵심 결과	판정
① 동일 예약 취소 중복 실행	30	releaseSeat() 호출 1회만 실행 (나머지 29건 먹등 리턴)	✓
② 취소 직후 재선점 경합	10	재선점 성공 1건 · SEAT_ALREADY_HELD 9건 · over-booking 0건	✓
③ 취소·해제 동시 호출	2	도메인 가드 위반(이중 취소) 0건 · 좌석 정확히 1회 반환	✓
④ REFUND_PENDING 구간 수량 초과	20	선점 성공 0건 · MAX_SEAT_COUNT_EXCEEDED 20건 전량 차단	✓

종합 결론

취소·재선점 동시 요청 수량 초과 0건, 좌석 상태 불일치 0건을 확인했습니다.

REFUND_PENDING·REFUND_FAILED 구간의 동시 요청 케이스도 RefundPendingQuantityGuardConcurrencyTest로 포함해 검증을 완료했습니다.

[Reservation] k6 결과 & 락 전략 3종 비교

분산락(Redisson)·비관적 락(SELECT FOR UPDATE)·낙관적 락(@Version+재시도)을 동일 시드·동일 시나리오로 비교합니다.

현재 채택

분산락 (Redisson)

장점 여러 서버에서도 정합성 보장
단점 Redis가 단일 장애점

비교 대상

비관적 락

장점 구현 단순, 별도 인프라 불필요
단점 DB 커넥션 점유시간 증가

비교 대상

낙관적 락

장점 경합이 적으면 가장 빠름
단점 경합 폭증 시 재시도 비용 급증

● 측정 완료 — 시나리오 A·B 각 1회, 3전략 정합성 100% 동일 확인 · 성능 비교는 반복 측정 필요

전략	A (성공/차단)	A p95	B (성공/50)	B p95
분산락 (Redisson)	1 / 49	1455.6ms	50 / 50	810.5ms
비관적 락 (SFU)	1 / 49	1348.4ms	50 / 50	1526.7ms
낙관적 락 (@Version)	1 / 49	1112.1ms	50 / 50	1645.7ms

[Payment] 결제 — 성능 테스트 정리 (k6)

서로 다른 주문의 신규 결제 동시 생성(A)과 동일 먹등키 반복 요청(B), 두 시나리오로 나눠 측정했습니다.

● 측정 완료 — POST /api/v1/payments · 시나리오 A(동시 신규 결제 생성) · 시나리오 B(동일 먹등키 재요청)

Trends

metric	avg	max	med	min	p90	p95	p99
http_req_blocked	3ms	4ms	3ms	1ms	3ms	4ms	4ms
http_req_connecting	1ms	1ms	1ms	315μs	1ms	1ms	1ms
http_req_duration	332ms	455ms	380ms	93ms	444ms	451ms	455ms
http_req_receiving	201μs	5ms	60μs	8μs	362μs	560μs	2ms
http_req_sending	47μs	343μs	24μs	5μs	117μs	220μs	311μs
http_req_tls_handshaking	0ms	0ms	0ms	0ms	0ms	0ms	0ms
http_req_waiting	332ms	455ms	380ms	87ms	444ms	451ms	455ms
iteration_duration	336ms	459ms	384ms	94ms	448ms	455ms	458ms

Counters

metric	count	rate
data_received	62.7 kB	23.3 B/s
data_sent	44.2 kB	16.5 B/s
http_reqs	100	0.04/s
iterations	100	0.04/s
payment_creation_created_count	64	0.02/s

시나리오

VU · 요청 수

성공률

p50

p90

p99

A · 동시 신규 결제 생성

100 · 100건

64%

380ms

444ms

455ms

B · 동일 먹등키 재요청

100 · 1,000건

100%

39ms

107ms

190ms

핵심 발견 — 먹등키 유니크 인덱스 갭 락 데드락

서로 다른 신규 결제 100건을 동시에 생성하자 성공률이 64%로 떨어졌습니다. 존재하지 않는 먹 등키를 SELECT FOR UPDATE로 조회하면 MySQL이 uk_payments_idempotency_key 인덱스에 갭 락을 거는데, 여러 트랜잭션이 같은 구간을 잠근 채 INSERT를 시도하면서 순환 대기(데드락)가 발생해 한쪽이 강제 롤백됐습니다.

후속 조치

결제 생성 트랜잭션만 READ_COMMITTED로 낮추는 방안을 검토 중입니다. SELECT FOR UPDATE 행 잠금·유니크 제약·주문별 Redis 분산락이 이미 이중으로 정합성을 지키고 있어, 격리 수준을 낮춰도 데이터 안전성은 유지될 것으로 보입니다.

06

자체 평가 의견

기획 의도와의 부합 정도 · 달성도 · 완성도 · 배운 점

자체 평가 의견

잘한 점

- Git Flow, merge 정책, PR·이슈 템플릿을 초반부터 합의·준수해 develop 히스토리를 깨끗하게 유지
- 설계 판단의 배경과 트레이드오프를 문서로 남겨 이후 코드 이해와 더블 체크 시간 단축
- 모니터링(T3-10)을 다른 기능보다 먼저 연결해 이후 변경의 비교 기준선 확보
- 락 전략 선택을 감이 아니라 비교 실험 데이터로 근거화, 동시성 테스트를 CI에 상시 편입
- 취소 이력을 PG 호출 전에 선기록해 이중 환불을 구조적으로 차단
- 티켓 상태를 6종으로 세분화해 환불 중·환불 실패를 명확히 구분

아쉬운 점

- 결제→주문→좌석 HOLD처럼 깊게 연결된 도메인의 인터페이스 계약을 초기에 확정하지 못해 통합 단계에서 흐름이 끊기는 상황이 반복됨
- 리뷰 타이밍을 정하지 않아 먼저 본 팀원이 승인·머지하는 패턴이 반복되며 다른 팀원의 리뷰 기회가 줄어들음
- 동시성 테스트가 방치된 원인을 사후에야 파악해 조기 발견 체계가 부족했음
- 락 전략 3종 비교를 일정 후반에 몰아서 진행해 반복 측정 여유가 적었음
- 혼잡도·마이페이지 등 3차 신규 기능의 자체 부하 테스트는 아직 진행하지 못함

Re:Seat

감사합니다

Q & A

TEAM 3조 · Re:Seat

github.com/Back-To-Back-team-3/re-seat · re-seat.netlify.app